RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer

Mehrad Haghshenas
University of Waterloo
David R. Cheriton School of Computer Science
Waterloo, Canada
m3haghsh@uwaterloo.ca

Meng Xu
University of Waterloo
David R. Cheriton School of Computer Science
Waterloo, Canada
meng.xu.cs@uwaterloo.ca

Abstract

Conformance testing asks whether an implementation agrees with its specification. When the specification is expressed in prose, one established approach is to mechanize it as an executable specification. This executable then serves as the oracle, and an input on which an implementation disagrees with it is a potential bug, either in the implementation or in the mechanized specification itself. How those inputs are obtained matters, because an oracle can only adjudicate the inputs it receives. Inputs can be derived from the specification itself, and coverage-guided generators can drive aggregate coverage high. But aggregate coverage does not provide a way to target a specific condition in the specification and generate an input that reaches it. This matters particularly for error conditions, where implementations are known to diverge and where the code is empirically under-tested.

This paper presents RuSMT, a framework for encoding prose specifications as executable oracles using a domain-specific language embedded in Rust. Its DSL provides types denoting Z3 sorts and primitives corresponding to Z3's operations. Concretely, the author writes the specification and marks a branch with a named marker, which identifies a path condition for the backend test synthesizer. In this paper, markers always identify error conditions. The same program is therefore used in two ways: compiled by `rustc`, it executes as the conformance oracle; lowered to SMT-LIB it yields one reachability query per marker. Z3 then solves each query: a `sat` model is rendered by a printer into a test program in the specified language, while `unsat` indicates that the marker is unreachable. If Z3 returns `unknown` or times out, RuSMT invokes a language model to propose a concrete candidate based on the emitted SMT-LIB and Z3's previous verdicts. The candidate is added to the original query as a single equality, thus constraining the search. If the candidate is rejected, its result is fed back to the language model to propose a new candidate. The process repeats until the fixed budget is exhausted.

We write two executable specifications in the DSL: IMP, Winskel's canonical small imperative language as a small end-to-end demonstration, and a TOML 1.1.0 parser, implemented from the published specification. We mark 2 branches in the IMP program and 183 in the TOML parser. Z3 alone solves both IMP queries. It solves none of the 183 TOML queries within our available computational budget: lowering the recursive parser as a whole effectively turns the problem into bounded model checking, whose formula-size blowup is well known. With the language model in the loop, RuSMT reaches 146 of the 183.

We ran the TOML-generated test suite against four independent TOML parsers and found divergences on 12 inputs. Of these, 9 test behaviour the specification leaves open, such as integer width and float exponent range, so they are not conformance obligations. The remaining three are implementation defects, 1 of which was previously unreported. The unreported defect was found in `smol-toml`, which accepts an *array-of-tables* header closed by a single bracket; we have reported this issue upstream.

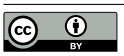
CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; *Formal software verification*; • **Theory of computation** → *Automated reasoning*.

Keywords

executable semantics, conformance testing, differential testing, SMT, bounded model checking, large language models

ACM Reference Format:
Mehrad Haghshenas and Meng Xu. 2026. RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer. In *Proceedings of the 1st International Workshop on Specification-Driven Development Life Cycle (SpecOps '26), October 4–9, 2026, Oakland, CA, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3842652.3843196



SpecOps '26, Oakland, CA, USA

ACM ISBN 979-8-4007-2970-6/2026/10
https://doi.org/10.1145/3842652.3843196

1 Introduction

Conformance testing asks whether an implementation agrees with its specification. When the specification is expressed in prose, one established way to make the question executable is to write the specification as a program. Systems such as K [26], PLT Redex [8], and Ott/Lem [21, 29] follow this approach. The executable specification is then the oracle: run it and an implementation on the same input, and a disagreement indicates a potential bug, either in the implementation or in the mechanization if it is not faithful to the prose.

An oracle only adjudicates the inputs it is given, so the next challenge is generating those inputs. Inputs can be generated from the specification itself [22], and coverage-guided generation drives aggregate coverage high [15]. But coverage is an aggregate measure and provides no mechanism for specifying a particular condition and generating an input that reaches it. This is particularly relevant for error conditions, where the code is empirically under-tested. In a study of production failures in distributed data-intensive systems,

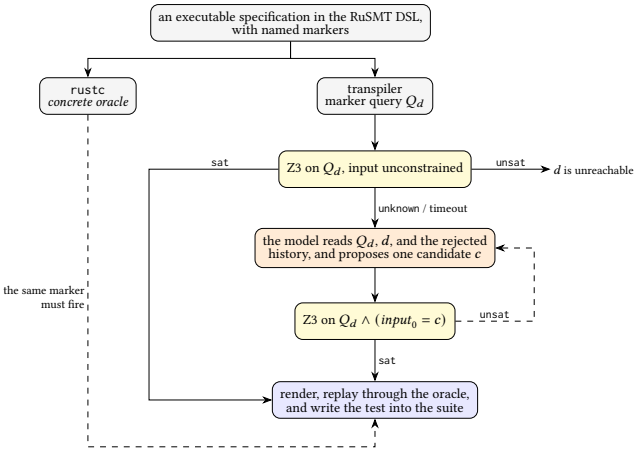


Figure 1: One executable specification and the loop that turns one marked branch into one test.

92% of catastrophic failures resulted from incorrect handling of non-fatal errors, and 58% could have been caught by simple tests of the error-handling code [39]. Error paths are also routinely dropped in Linux file systems [9]. Error conditions are also a common source of implementation divergence. For example, no two JSON parsers agree across malformed documents [28], and discrepancies between parsers on ill-formed input are a documented source of security vulnerabilities [3, 27]. Error behaviour is part of normative conformance, as reflected in conformance suites such as Test262 [7].

RuSMT targets cases where the author wants to name a condition and obtain an input that reaches it. The specification is written in a subset of Rust whose types denote Z3 sorts and whose primitives denote Z3's operations [6]; compiled, it is the oracle, and lowered to SMT-LIB, it becomes the input to the solver. Concretely, the author marks each path condition with a marker, and the transpiler emits one reachability query for each of them. If the query is satisfiable, Z3 returns a model, which a printer renders as a test program that reaches the path condition. If the query is unsatisfiable, the condition is unreachable. If Z3 returns `unknown` or times out, a language model proposes a concrete input. Z3 checks the proposal by adding it as an equality constraint to the original query. The language model therefore proposes candidates; Z3 remains responsible for validation. If the constrained query is unsatisfiable, the result is fed back to the language model, which proposes another candidate; this continues until the fixed budget is exhausted. Figure 1 shows the flow.

Contributions. This is a tool-and-artifact paper. We propose no new solver, synthesis algorithm, or completeness result. Related techniques have been studied previously in conformance testing from a formal specification [11], solver-aided synthesis [30], and counterexample-guided inductive synthesis with a language model as the candidate generator [12].

(1) A framework for writing one recursive program that serves as the conformance oracle when compiled and is lowered to SMT-LIB, from which a conformance test suite is generated with one test for each marked path condition for which Z3 finds a model.
(2) Two executable specifications written in it: IMP, with 2 markers, and a TOML 1.1.0 parser with 183. Z3 alone answers both IMP queries and none of the TOML ones within the available budget; with the language model in the loop the TOML suite covers 146 markers (§5, §6).
(3) A run of the TOML suite against four independent parsers. On 12 inputs the parsers and our oracle did not all agree: 9 exercise behaviour the specification leaves open, while three are implementation defects, 1 of them previously unreported and now filed (§6).
(4) A negative result on solver-alone synthesis. Lowering a recursive parser whole-program is bounded model checking [2, 5]; we report how sharply its known blowup arrives on a real parser (§3, §6).

Research questions. RQ1: How many marked path conditions does the pipeline produce tests for, and how many can Z3 produce tests for alone? *RQ2:* How often does a proposed input fail to reach the path condition it was proposed for; i.e., how often would the language model propose an input that does not satisfy the target? *RQ3:* When the generated suite is run against independent implementations, on how many inputs do the implementations and the oracle disagree, and what does each disagreement turn out to be?

Together, these questions ask whether the pipeline can generate targeted tests, whether those tests can be trusted, and whether they reveal conformance discrepancies in independent implementations.

2 The RuSMT DSL

A program written using the RuSMT DSL is embedded in Rust and serves two purposes: compiled by `rustc`, it executes as the conformance oracle; lowered by the transpiler, it becomes an SMT formula. The DSL sits between two bounds. The upper bound is imposed by SMT: every construct admitted by the DSL must be translatable to a corresponding SMT encoding. The lower bound is imposed by expressiveness: the DSL must remain expressive enough to write a whole interpreter. The resulting fragment sufficed for both case studies in §5. In this section we describe the fragment, the standard library that gives it its types, and the marker type on which the synthesis pipeline is built.

The fragment. The parser of the transpiler accepts a fixed set of expression forms and rejects every other Rust expression. The accepted forms include literals (integer, floating-point, boolean, and string), variables, construction of `structs`, `enums`, and tuples, field access, function calls and method calls, `if` expressions with a mandatory `else`, `match`, blocks and parenthesized expressions, and the dereference `*` of a `Boolean` (which is how a branch condition is read). Three macros provide bounded quantification and choice over a collection: `forall!`, `exists!`, and `choose!`. A function body consists of let bindings followed by a single expression as the return value.

The fragment excludes loops, assignments, closures, references, and indexing. Without mutation and references, a value never aliases and never changes under another name, so a `let` is a substitution and the translation is referentially transparent; this is the same discipline that lets a big-step operational semantics be read

as a system of equations. Without loops, recursion is the only form of iteration.

At the type level, the transpiler accepts `structs` with named or tuple fields and `enums` whose variants may be unit, tuple, or struct variants. It rejects, in particular, unit structs, default values, and any other attributes. Type parameters are permitted provided that they are bounded only by the library's `SMT` trait, which bridges the gap between the Rust type system and the SMT solver. At the function level, `const`, `async`, `unsafe`, explicit ABIs, and variadic parameters are rejected. A specification outside the fragment therefore fails to compile rather than being mistranslated. These restrictions are enforced by the `#[smt_type]` and `#[smt_fn]` attributes, which are applied to each type and function that the transpiler accepts.

Errors as values. The fragment has no `panic!`, `?`, or `.unwrap()`, so every departure from the expected path that a specification expresses is represented as a returned value rather than an exception. A DSL function can nevertheless panic at run time through a library primitive called outside its documented precondition; the rule for the standard library given below governs that case, and the interpreter is expected to guard such calls.

The standard library. The DSL's library types are lowered to Z3 sorts, and their methods to Z3 operations; Table 1 lists these mappings. The scalar types map onto Z3 theories, namely `Boolean`, the unbounded `Integer` and `Real`, the bit-vectors `I32`, `I64`, `U32`, and `U64` (where signed and unsigned are two readings of one Z3 sort, distinguished by the operations applied to it), and the IEEE floating-point types `F32` and `F64`. `String` and `Seq<T>` map onto the sequence theory. `Set<T>` and `Array<K,V>`, however, are not represented directly using SMT arrays, since that theory has no notion of size; each is lowered to an algebraic datatype that pairs an SMT array with an explicit cardinality (and, for `Array`, with a second array recording which keys are present), so that `length` and `is_empty` become definable. `Path` is the marker type and is described below. `Cloak<T>` is a `Box`-like indirection that exists only so that an author can write self-referential types under Rust's sized-type rule; the intermediate representation erases it, lowering every `Cloak<T>` to `T`, because Z3 supports recursive datatypes natively.[1] Every method in the table is registered in the transpiler as an *intrinsic*, i.e., a fixed mapping from a method call to the SMT term that the backend emits for it, and this registry is the only place where a Rust method acquires an SMT meaning.

Two layers are thus kept apart. The standard library denotes Z3's theories, whereas the executable specification encodes the semantics of the target language. The particularities of a target live in the interpreter as explicit guards, and never in the library. Thus, the correctness of the library's Rust-to-SMT translations needs to be established only once, for a fixed set of primitives, rather than separately for each case study. Concretely, a primitive denotes Z3's operation, not Rust's and not the target's, and where the Rust and SMT readings differ, the library documents the difference. For a primitive f, exactly one of the following holds.

(1) **Total agreement.** Rust and Z3 agree on every value in the primitive's domain, and f is used directly.

[1]Provided that the type also has a non-recursive variant; otherwise the lowered datatype is uninhabited.

Table 1: The standard library: DSL types, the Z3 sorts they denote, and their operations.

Type	Z3 sort	Operations
`Boolean`	`Bool`	`and`, `or`, `not`, `xor`, `implies`, `iff`, `ite`
`Integer`	`Int`	arithmetic, comparisons, radix decoding (`from_hex_str`, …), range tests (`is_gt_i64_max`, …), conversions
`Real`	`Real`	arithmetic, rounding, comparisons, conversions
`I32`, `I64`, `U32`, `U64`	`(_ BitVec 32)`, `(_ BitVec 64)`	`bv_add`, …, `bv_rem`, comparisons, bitwise operations, shifts, rotations
`F32`, `F64`	`(_ FloatingPoint 8 24)`, `(_ FloatingPoint 11 53)`	IEEE arithmetic, comparisons, predicates, rounding, conversions
`String`	`String`	`at`, `concat`, `contains`, `starts_with`, `index_of`, `substr`, `replace`, `to_int`, `from_int`, `to_code`, `from_code`, …
`Seq<T>`	`(Seq T)`	`at`, `extract`, `concat`, `append`, `unit`, `contains`, `prefix_of`, `suffix_of`, `index_of`, `replace`, `length`
`Set<T>`	datatype over `(Array T Bool)` with cardinality	`insert`, `remove`, `contains`, `is_subset`, `is_disjoint`, `length`
`Array<K,V>`	datatype over `(Array K V)` with presence and cardinality	`store`, `select`, `del`, `contains_key`, `length`
`Path`	`(_ BitVec N)`, one bit per marker	`named`, `merge`
`Cloak<T>`	erased to `T`	`shield`, `reveal`

(2) **Agreement under a precondition.** They agree on a documented subset of the values and differ outside it, in one of two ways. In the first, Z3 defines a value where Rust panics: division by zero is defined in SMT-LIB (`bvudiv` returns all ones, and `bvsdiv` returns all ones for a non-negative dividend and 1 for a negative one), whereas the corresponding Rust division panics. In the second, the SMT term is well-formed but the theory does not constrain its value: `(div x 0)` over the integers is a legal term, and with $x = 7$, for example, Z3 can satisfy `(= (div x 0) 100)` and `(= (div x 0) 999)` alike.[2] In either form, the *interpreter* guards the differing inputs and calls f only on the rest.

Listing 1 is the division guard of the IMP arithmetic evaluator. The interpreter branches on a zero divisor. The zero-divisor branch is marked because it is a condition the specification identifies and for which we want to synthesize an input; the other branch calls `bv_div`, which agrees with Z3 for every nonzero divisor and therefore requires no guard. The code is otherwise ordinary recursive Rust; the only token specific to synthesis is `Path::named`. From this source, RuSMT synthesizes `A := (0 / 0)`, and replaying the program through the same `eval_aexp` reaches the marker.

```
if *new_r.eq(I64::from(0)) {
    ArithmeticResult::Err(Path::named("division_by_zero"))
} else {
    ArithmeticResult::Ok(new_l.bv_div(new_r))
}
```

Listing 1: The division guard of IMP's evaluator.

Markers. A *marker* identifies a path condition for which the author wants a test to be generated. Concretely, a `Path` is a set of marker IDs, and `Path::named` creates a singleton marker. An execution that trips several markers accumulates them, and `Path::merge` computes their set union.

[2]Both checked on Z3 4.15.4. The second form, if left unguarded, would make the framework unsound: the solver could choose an unspecified value that satisfies the marker assertion even though no corresponding Rust execution exists.

A marker must have the same identity in the compiled evaluator and in the SMT encoding so that the test generated for it can be checked against the same condition during replay. Accordingly, `Path::named("d")` derives the marker ID by hashing the name d, and the same hash function is used by the transpiler and the compiled evaluator. This allows replay to check that the named marker was actually reached. In the SMT encoding, a `Path` is a bit-vector with one bit per named marker in the program, and the set operations on the type become bit-vector operations:

$$\emptyset \mapsto \texttt{(\_ bv0 N)}, \qquad \{m\} \mapsto \text{one-hot}(m),$$
$$l \cup r \mapsto \texttt{(bvor } l\ r\texttt{)}, \quad T \subseteq s \mapsto \texttt{(= (bvand } s\ M_T\texttt{) } M_T\texttt{)},$$

That is, the empty path is the zero vector, a named marker sets its own bit, `merge` is bitwise or, and "the path s contains every marker of the target T" is the masked equality on the right, where M_T is the vector with exactly the bits of T set.

3 From a Marker to a Conformance Test

The annotated source is compiled in two modes. In the first, `rustc` compiles the interpreter, which then executes on concrete inputs. In the second, the transpiler parses the fragment of §2, lowers it to a sort-checked intermediate representation by Hindley–Milner unification over SMT sorts. It further monomorphizes generic functions on demand, records every `Path::named` site as a target, and emits one query per target. Each query is a self-contained SMT-LIB2 file. The driver then runs the loop of Figure 1 over these files and writes the accepted witnesses out as the suite. Adding a language means writing an interpreter using the DSL.

3.1 Markers to Queries

The entry function of a specification takes an input parameter. For example, the IMP specification takes an abstract syntax tree representing an IMP program, while the TOML specification takes a TOML document as an input string. To generate a test for a path condition, RuSMT treats this input as symbolic and asks Z3 whether some assignment to it causes the executable specification to reach the target. Any other state used by the specification is fixed to the same value in both the symbolic and concrete executions. For example, the IMP entry function `eval_com(c: Com)` fixes the empty store internally in the interpreter, so the program is the single free input. If the store were instead left symbolic, Z3 could choose an initial state that is never used by the concrete specification, producing a witness that does not replay. The TOML entry point `parse_toml` takes only the document as an input, so the document is the only symbolic input and no additional input needs to be fixed.

For a path condition P, which is a set of marker IDs, RuSMT generates an SMT query that declares one symbolic constant for each parameter of the entry function (`input_0`, …), applies the entry function to these constants, and asserts that the resulting `Path` contains every marker in P. Any code-point parameter is additionally bounded to the solver's alphabet (see §8). Thus, the query asks

$$\exists x.\ \mathit{entry}(x) = \mathit{Err}(p) \ \wedge\ P \subseteq p,$$

where x is the symbolic test input and p is the `Path` accumulated during execution. If the query is satisfiable, Z3's model assigns concrete values to x, which are rendered as a test.[3] A singleton ($P = \{i\}$) corresponds to a `Path::named` marker; a larger P arises from a `Path::merge` site and asks for one input that reaches all markers in the set. We use containment rather than equality because an execution may reach additional markers without invalidating the target condition.

A recursive interpreter is not a finite formula. RuSMT provides a flag `k=N` that controls how recursive calls are encoded. With `N=0`, which is the default and what both case studies use, the recursive strongly connected components of the call graph are emitted as `define-funs-rec`, and the solver handles the recursion natively. With `N` ≥ 1, every recursive component is unrolled into N+1 depth-indexed copies $f_0, \ldots, f_N$, where f_d calls f_{d-1} for callees in the same component, f_0 is a terminator, and external callers enter through f_N. Under depth-N unrolling, a `sat` model is a witness that is reachable in at most N recursive steps, and this is the sense in which the completeness is *bounded*, as in bounded model checking [2].

Unrolling has two consequences. First, a bounded `unsat` means only that no witness exists within the unrolling depth k; it does not show that the marker is unreachable. For example, in IMP, at `k=1` the query for the division-by-zero target returns `unsat`, whereas the same query is `sat` at `k=0`. For this reason, the pipeline treats `unsat` as a definitive verdict only at `k=0`, where no unrolling is applied. Second, the terminator (f_0) must return a value. RuSMT gives it a nullary *bottom* variant that the concrete semantics never produces. Consequently, the artificial cutoff cannot satisfy an `Err`-marker assertion, and the bottom value cannot appear in a concrete execution.

More generally, we claim no completeness in the absence of resource and depth bounds. Even when an input reaching a marker exists, Z3 may return `unknown` or time out, since the encoded formula can fall outside what the solver decides within its resource limits. Thus, any completeness provided by the pipeline is bounded by the recursion depth and contingent on the solver.

3.2 Solving, and Where the Solver Stops

The query is first handed to Z3 exactly as emitted, with the input unconstrained. For IMP, this is the whole story; both markers are discharged unaided, in 220 ms and 109 ms, respectively. We treat a `sat` result as providing a valid witness and an `unsat` result as establishing that the target is unreachable under the unbounded encoding. An `unknown` result or timeout provides neither, and therefore costs coverage.

For TOML, the solver never solved a query within our computational budget. The reason is the size of the formula produced when the recursive parser is lowered as a whole. This is bounded model checking [5]: recursive calls are unrolled into a finite formula that represents executions up to the chosen depth, and the resulting formula can grow rapidly with the number of recursive calls [2, 5]. In our encoding, the growth is particularly severe because the parser operates on a symbolic input. With a concrete document, preprocessing can simplify each conditional as the input flows through the parser. With a symbolic document, however, the solver cannot

[3] The renderer extracts the input parameters to the entry function, inlines `let`-shared subterms, maps mangled constructors back to AST nodes, canonicalizes solver-chosen identifiers, and decodes bit-vector literals as two's-complement signed integers.

determine which branch will be taken, so both arms remain in the formula and the expansion multiplies through the nesting.

The difference between concrete and symbolic inputs is stark on the real TOML parser. Taking the query for one marker and varying only how much of a 16-character document is left free, the query with all 16 characters fixed by equality constraints is decided in 798 ms, whereas leaving a single character unconstrained is already enough for Z3 to return `unknown` at 30 s; two, three, four, six, and eight free characters behave identically. A single character ranges over roughly 55k values, so this is not an effect of the size of the search space. The effect instead comes from the expansion: any free variable at all defeats the folding, and the whole parser stays live regardless of how tightly that variable is constrained. Consequently, there is no partially symbolic middle ground to exploit here; the concretization step that makes concolic testing work [4, 24] has no foothold, because the formula never becomes small. We also tested emitting everything as `define-fun-rec`, so that unfolding becomes lazy; this made matters worse, since the character-class predicates could then no longer fold either.

3.3 The Language Model in the Loop

The design of the loop follows from this diagnosis. If Z3 is effective once the input is concrete but struggles to find an input symbolically, then the productive move is to obtain concrete candidates by other means and let Z3 validate them. Accordingly, when Z3 returns `unknown` or times out, the driver invokes a language model, which receives the emitted SMT-LIB text, the name of the marker, and the candidates already rejected together with Z3's verdict on each, and which returns one candidate. When the language model supplies a candidate c, RuSMT adds the constraint that the parameter of the entry function equals c to the original query Q_d, producing $Q_d \wedge (input_0 = c)$. If Z3 returns `sat`, the candidate therefore satisfies the original query as well: every model of the strengthened query is also a model of Q_d. On the other hand, an `unsat` answer says only that this particular candidate does not reach this particular marker, and it is fed back as the context of the next round. Each marker is given the same number of rounds.

When a proposed input candidate from the language model is rejected by Z3, the pipeline also determines which marker(s) the candidate actually reaches. Specifically, rather than returning a bare `unsat` to the language model as feedback, the pipeline gives a second query to Z3, with the same input pinned, but without the original marker assertion. It instead asks Z3 to evaluate the `Path` returned by the entry function, thereby identifying the marker(s) actually reached by the candidate. The following example illustrates why this matters: the TOML reference raises `array_table_missing_close_char` when an array-of-tables header is closed by something other than `]]`. Z3 returned no verdict on the unconstrained query for this marker, as for every other TOML marker, so the language model was shown the SMT-LIB, the name, and an empty history, and it proposed `[[a`. Pinned into the query, that document gave `unsat`; the second query showed that it reaches `array_table_missing_close_eof` instead, since the header runs into the end of the input before any closing bracket. That answer was fed back, and in the next round the model proposed `[[a]x`, which Z3 accepted. The input was then re-executed through the compiled semantics, where it fired the same marker, and it entered the suite. In §6, one of the four TOML parsers we test mistakenly accepts this document as valid.

Note that nothing in the design specifically requires a language model. The proposer is an ordinary shell command that reads a prompt on standard input and writes a candidate on standard output, so an enumerator or a fuzzer is a drop-in replacement. The acceptance step, and with it the guarantee of §4, is unchanged by the substitution. We did not evaluate an undirected generator. In our experiments, the numbers in §6 show that Z3 rejected about three in four proposals even from a marker-targeted model, mostly because an earlier error masked the intended marker; an undirected generator would have to discover the relevant marker as well as produce an input that reaches it.

4 RuSMT's Trusted Computing Base

The guarantees of RuSMT depend on the correctness of several components. First, the executable specification: note that the implementation of the prose specification in the RuSMT DSL is written manually, and we provide no guarantee that it faithfully transcribes the standard. Second, the correspondence between the standard library's primitives and their Z3 encodings. Third, the transpiler that lowers the DSL to SMT-LIB. Fourth, the concrete parser and pretty-printer used for replay. Furthermore, the Rust compiler and Z3 engine are trusted. A bug in any of these components can affect our results.

The per-construct correspondence. The Rust implementation and SMT encoding of each standard-library primitive must agree on the inputs for which the primitive is used. We state this requirement as a per-construct correspondence.

Definition 4.1 (Per-construct correspondence). For every stdlib function f and every concrete input x satisfying the documented preconditions of f,

$$rust\_impl(f, x) \;=\; z3\_eval\big(z3\_formula(f),\, x\big).$$

We *assume* this correspondence rather than prove it formally, although we also checked the primitives by differential testing. We make the correspondence auditable: the documentation of each stdlib method states the exact Z3 formula that the backend emits for it, together with its preconditions and, where the two readings genuinely differ, the inputs on which they differ. In case of divergence, the author of the interpreter must guard the corresponding inputs before calling the primitive. This is the division of labour of §2: a primitive that matches the target exactly is used directly, while a primitive that diverges on specific inputs is guarded on those inputs. Because concrete Rust and Z3 take the *same* branch on the same input, the equality of Definition 4.1 is preserved.

Replay. A candidate is accepted for a marker only if the executable specification, given the same candidate input, reaches the same named marker that Z3 reported. Since acceptance is already a logical consequence of `sat`, a discrepancy between the symbolic and concrete executions can arise only if Definition 4.1 fails on some construct that the candidate exercises. Replay is a check on the lowering and catches precisely that disagreement. A defect in the correspondence can therefore cause a spurious rejection, but

not a false acceptance. Likewise, replay rejects a spurious marker reached only through the artificial bottom value at the cutoff of a depth-bounded unrolling (§3).

5 Case Studies

RuSMT is meant for the authors of new executable specifications, and the two case studies were chosen to stress two different regimes.

5.1 IMP

IMP is the canonical small imperative "while" language of Winskel [36]. Its big-step semantics is familiar and recursive, yet small enough to demonstrate an end-to-end flow of the RuSMT pipeline. We added two dynamic error branches, namely `division_by_zero` and `undefined_variable`, so that the study answers whether a direct-style interpreter can be lowered, solved, rendered, and replayed without bounded unrolling and without a language model. Z3 synthesizes an AST rather than source text: the AST is more compact for Z3 than the complete source text. The source text is therefore parsed into an AST as a preprocessing step, and the printer described in §3 is exercised when rendering the generated tests. The specification is 1,084 non-blank lines of Rust.

5.2 TOML

TOML 1.1.0 [25] was chosen because it is large enough to expose the solver's scalability limitations, has a precise public specification, and has several independent implementations to test against. The executable parser, written from the published specification, has a single free input, i.e., a sequence of Unicode code points, and 158 functions covering key-value pairs, tables, arrays, inline tables, booleans, integers, floats, strings, and datetimes, in 5,878 non-blank lines. Its 183 named markers cover syntactic errors (malformed keys, unterminated strings, invalid escapes, and missing array or table delimiters) as well as semantic violations (integer and float overflow, out-of-range date and time fields, duplicate keys, and redefined tables). This is the case study that motivated the use of language models. The witnesses produced by the loop are also small and readable, although this was not an explicit design goal. This matters downstream because a small document makes it easier to isolate the rule it violates. We do not claim a verified TOML formalization. We claim a formalization with 183 named obligations, and a suite generated from it.

6 Evaluation

Setup. All measurements were taken on an Apple M1 running macOS 26 with Z3 4.15.4 and Rust 1.88.0, with Z3 driven as a subprocess over SMT-LIB2. Both suites were generated by the driver of §3. The language model was OpenAI's `gpt-5.4-mini`, invoked through the `codex` command-line runner as an ordinary shell command, with one witness requested per marker, nine rounds per marker, and a 20 s Z3 budget for each pinned candidate. The unconstrained query was given a budget of 2 s per TOML marker. Separate probing had already established that a symbolic TOML input does not decide at any budget we could afford, and that Z3 charges its `-t:` limit against CPU time rather than wall-clock time, so that a 60 s budget on one of these queries returned `unknown` only after 347.6 s of wall-clock time. A larger budget is therefore not practical; queries that do not decide within the short probe are passed to the language model.

Table 2: New witnesses per round over the 183 TOML markers.

Round	New witnesses	Cumulative
1	57	57
2	56	113
3	19	132
4	6	138
5	3	141
6	2	143
7	0	143
8	2	145
9	1	146

The round limit and per-candidate Z3 budget were fixed in advance and not tuned. A smaller budget can turn a marker that would otherwise be covered into a miss, but cannot create a witness (§8). We therefore report the effect of the round limit directly: the cumulative column in Table 2 shows coverage after each round. We also record every model exchange and key it by a content hash of the prompt. Because the model is nondeterministic, an independent rerun need not produce the same suite; our experimental artifact is the recorded run and the suite it produced.

RQ1: coverage, and the share of it that the solver produced alone. For IMP, Z3 returned a model for both markers under native recursion, in 220 ms and 109 ms, with no language model involved; the printer rendered them as `A := (0 / 0)` for `division_by_zero` and `v0 := v0` for `undefined_variable`, and both replayed. We do not run IMP through the differential harness, since it is a small end-to-end demonstration and not a language with competing production parsers.

For TOML, the lowering produces SMT-LIB that Z3 accepts without a front-end error, but the unconstrained solve produced no witness for any of the 183 markers, and no query returned `unsat` either, so no marker was shown unreachable. In short, none of the unconstrained queries produced a decisive result. With the language model in the loop, coverage went from 0/183 to 146/183. Table 2 shows how the witnesses arrived. The first two rounds account for the majority of them; the later rounds add progressively fewer. Thus, the reported coverage is the coverage achieved within our computational budget; the 37 markers without a witness remain unresolved rather than being ruled out. We did not fill the misses with hand-written tests.

The misses are not spread uniformly across the markers. 20 concern date, time, and numeric-offset markers; 10 are numeric-literal markers, mostly float overflow and malformed radix forms; 6 concern table or key redefinition and unterminated composite constructs; and 1 is an array case. The pattern is consistent with the structure of the parser: the loop handles local syntax errors well, but struggles when the intended marker is easily masked by an earlier parse error or when reaching it requires passing through a long chain of numeric or temporal sub-parsers.

RQ2: how often does the language model target the wrong marker? The per-round history was recorded for every marker. Over the run, the language model made 588 proposals that reached Z3 and got a verdict: 146 were accepted and 442 were rejected as `unsat`.

Table 3: Agreement of each implementation with the RuSMT oracle on the generated TOML suite. Every suite input is rejected by the oracle, so a divergence is an acceptance.

Implementation	Agreement	Divergences
Rust `toml 1.1.4+spec-1.1.0`	144/146	2
CPython `tomllib` (3.14.7)	137/146	9
BurntSushi `toml` v1.6.0 (Go)	142/146	4
Node `smol-toml` 1.8.0	142/146	4

Table 4: The 12 inputs on which the parsers and the oracle did not all agree. Group A is not an obligation of the specification and group B is already tracked upstream, so only group C was filed.

	Group	Inp.	Filed
A	Standard leaves it open	9	no
B	Implementation defect, tracked	2	no
C	Implementation defect, unreported	1	**yes**
	Total	12	1

A further 28 proposals repeated an already rejected candidate and were dropped before Z3, and on 19 occasions the language model returned no parsable candidate at all. No acceptance check timed out. In other words, about three of every four proposals that reached Z3 were rejected for the marker they were targeting. The solver's verdict therefore prevents these proposals from being accepted under the wrong obligation.

The second query of §3 shows what happened to the rejected proposals. Over the run, 88.2% of the rejections came back naming a different marker, 5.9% reported that the proposal was simply a valid TOML document, and on 5.9% the observation itself returned nothing, so the proposer received the bare `unsat`. The failures are systematic rather than scattered. The usual cause is that the proposed TOML document contains an earlier violation, so that the parser stops before the intended branch.

RQ3: what the suite found. We want to clarify that a disagreement between an implementation and the *oracle* is a conformance finding relative to the specification. The quality of our conformance testing, however, is only as good as our formalization of the specification. Implementations that disagree with *each other* are a differential-testing result [20], which shows that the generated suite discriminates between implementations.

We ran the 146 generated documents against four implementations, each the latest release at the time of writing: the Rust `toml` crate `1.1.4+spec-1.1.0`, the `tomllib` module of CPython 3.14.7, BurntSushi's `toml` v1.6.0 for Go, and `smol-toml` 1.8.0 for Node. Every document in the suite is rejected by the RuSMT oracle, so a divergence occurs when an implementation accepts it. Table 3 gives the per-parser counts. All four parsers agreed with the oracle on 134 of the 146 documents; on the remaining 12 inputs the parsers and the oracle did not all agree. Table 4 accounts for all 12 inputs, which fall into three groups.

Group A: the standard leaves it open (9 inputs). This is the largest group, and it contains no defect on either side. Six inputs are cases where the `tomllib` module accepts integers outside the 64-bit signed range. TOML states that "implementations are free to support any integer size" [25], recommends at least 64-bit signed, and requires an error only when a value "cannot be represented losslessly". Our reference uses `i64`, so `18446744073709551616` genuinely cannot be represented and we *must* error; the arbitrary-precision integers of CPython represent it exactly, so CPython need not. Both are compliant with the specification. The other three inputs concern our `i32` bound on the float exponent, and here the specification says only that implementations are "free to support any precision level" [25], and it states nothing about exponent range. Where the specification imposes no requirement, neither behaviour can be considered non-conforming. 9 of the markers in the suite (15 in the catalog as a whole, once misses are included) name branches that exist because of a representation *we* chose, and not because the standard distinguishes them. They are legitimate branches of our semantics, but they do not define conformance obligations.

Group B: tracked defects (2 inputs). BurntSushi accepts `a = { b = 1 }\na.c = 2`, which extends an inline table that the standard calls self-contained, and `a.b = 1\n[a]`, where a header redefines a table that a dotted key created implicitly. Unlike group A, the standard is explicit in both cases, and both are prohibited by the specification. Both are also already known to the maintainers: the project's `toml_test.go` skips `invalid/table/redefine-02`, `redefine-03`, and `invalid/inline-table/overwrite-02` beneath the comment "`TODO: fix this; we allow appending to tables, but shouldn't`". Our suite reached them from the specification rather than from the test corpus, which is a check on the method; nevertheless, they are not new, and we do not file them.

Group C: the unreported defect (1 input). We found one previously unreported defect in `smol-toml` 1.8.0: it accepts `[[a]` even though an array-of-tables header requires two closing brackets. The generated witness was `[[a]x`, which minimizes to `[[a]` and is parsed by `smol-toml` as `{"a":[{}]}`. The other three parsers reject it. Note that `smol-toml` passes all five `toml-test` cases for unclosed table headers, but each places another line after the malformed header, so none exercises the end-of-input case. Thus, passing the existing tests does not imply that this branch is covered. The TOML 1.1.0 grammar explicitly requires two closing brackets through `array-table-close = ws %x5D.5D`, confirming the divergence. We reported the defect upstream and contributed the missing cases to `toml-test`.[4]

7 Related Work

Solver-aided executable semantics. Several systems combine executable semantics with solver-based reasoning. Rosette [32, 33] is the closest general-purpose host: it symbolically executes supported language constructs while evaluating the rest concretely, allowing client analyses to use the resulting constraints. SyGuS [1] and SemGuS [14] instead synthesize the program itself, against a grammar or a semantics supplied separately, while Synantic [17] synthesizes semantics from a black-box interpreter. RuSMT keeps both the program and its semantics fixed: the unknown is the input

[4]Reported as `squirrelchat/smol-toml` issue #65; the two missing cases were also contributed to `toml-test` as pull request #205.

to the executable specification. The solver is used to find an input that reaches a marked path condition in that specification.

SCALE [13] is the closest prior system to this approach. It encodes executable models of the DNS RFCs in Zen, a solver-backed DSL embedded in C#, and symbolically executes them to generate zone files and queries, which are then run against eight nameservers. The key difference is how tests are selected. SCALE explores the model by enumerating bounded scopes, whereas in RuSMT we target error conditions for which tests should be generated, producing one witness per marked branch.

Specification-derived testing. Once a specification is executable, it can serve not only as an oracle but also as a source of test requirements. K [26], PLT Redex [8], and Ott/Lem [21, 29] derive executable tools from formal rules and specifications. JEST [22] is closest in purpose to RuSMT: from a mechanized ECMAScript specification, it derives an oracle, a test synthesizer, and a conformance tester. Its successor introduces feature-sensitive coverage [23], which refines test requirements using the innermost enclosing language feature. Our test requirements are instead explicit error conditions in the executable specification: a marker identifies the branch for which RuSMT should generate a test. Thus, the granularity of the generated suite is determined by marker placement rather than by aggregate coverage or the syntactic structure of the specification. Csmith [38] likewise targets differential testing, but generates programs from a grammar.

Conformance testing from formal specifications. Deriving a conformance suite from a formal specification is a mature goal; TGV [11] synthesizes conformance tests from specifications of non-deterministic reactive systems. What we change is the packaging. The specification is an ordinary Rust program rather than a separate formalism, so the oracle is a compilation of the same source and there is no second artifact to keep in step with it. The coverage criterion travels with it: the markers are written inline in the semantics rather than maintained as a coverage model beside it.

Whole-program versus per-path encodings. Encodings divide by granularity: one query for the whole program, unrolled to a bound, or one query for each explored path. Our unconstrained query is on the first side, which is bounded model checking [5]. Symbolic and concolic executors [4, 24] are on the second, and that is why they do not meet the wall we report in §3.2: each of their formulas covers one path, so the cost moves from formula size to path explosion [4]. Language models have been attached to both sides. Cottontail [34] drives concolic execution over structured inputs, and AutoBug [16] splits the analysis path by path and asks a model for the reasoning a solver would perform. In both, the model reads the program. In RuSMT we give it the emitted SMT-LIB and Z3's verdicts.

Language models behind a solver. The propose-then-check pattern is not new either: it is CEGIS [30] with a language model in the generator position [12]. What varies is what the model proposes. Lemur [37] proposes verification structure; AquaForte [18] proposes SMT instantiations that the solver rechecks; CodaMOSA [15] and Cottontail [34] propose test inputs once conventional generation stalls. RuSMT proposes one input and adds it to the original query as an equality, so the obligation Z3 discharges is still the marked error condition. The language model never widens what counts as success.

8 Limitations and Threats to Validity

Validity depends on the formalization. The validity of a conformance result is relative to the executable formalization. In other words, the formalization must faithfully implement the specification's behaviour; if it does not, an input may produce a different result in the formalization than it should under the specification, and a conformance test based on that input can therefore report a false divergence or miss a real one. The coverage of the generated test suite, however, depends on the markers in the formalization. If an error branch is not marked, RuSMT cannot generate a test targeting that branch, and an implementation that accepts an input exercising that error may therefore go undetected. Conversely, if a marker is attached to a branch whose behaviour is permitted by the specification, the resulting test can report an implementation as non-conforming even though its behaviour is permitted by the specification.

A miss is not a proof of unreachability. A marker reported as uncovered is one for which no input was *found*, not one for which none exists. Only an `unsat` on the unmodified, unbounded query would establish the latter, and the unconstrained solve returned no verdict on the TOML queries. We wrote every marker to be reachable and expect none of the 37 misses to be a dead branch, but the pipeline cannot confirm this.

One witness per marker. A marker identifies a specific condition in the executable specification, and the suite contains one witness that reaches that marker. The markers are fine-grained enough that distinct conditions are represented by distinct markers; for example, the TOML catalog uses separate markers for different error conditions. Thus, multiple witnesses for the same marker are not needed and we use one witness per marker in the reported run, which also limits the number of model calls across the suite. The driver nevertheless supports multiple witnesses per marker by adding a blocking assertion after each accepted witness and re-solving.

The authoring cost. The author writes one Rust program, of 1,084 non-blank lines for IMP and 5,878 for the TOML reference, and marks the error branches in it; from that single source, the framework derives the oracle and the conformance suite. The cost is therefore that of writing the executable specification once. It is real, and it is what the authoring mode below aims to reduce.

Authoring the specification with a language model. The framework also includes an authoring mode that uses a language model to draft a specification from a prose standard. Each generated specification is checked for validity, and failures are fed back to the model to guide subsequent drafts. This propose-and-check workflow follows the approach of AutoSpec [35] and SpecGen [19]. We do not evaluate this mode in the present work. Instead, we provide the harness as a basis for future evaluation, where the model is given the TOML standard and the DSL, but neither our reference implementation nor its marker names. Its output can then be compared with the reference on two dimensions: whether it agrees on

acceptance and rejection of valid and invalid TOML documents, and how many distinct error markers it recovers. The complete harness is included in the artifact. Note that the final draft still requires human review, since the model can misread the standard and produce a specification that is not faithful to it.

The solver frontier. The dominant limitation is the whole-program encoding. Because the entire semantics is lowered into a single query, the solver must reason about all paths through the program at once. The measurements in §3.2 show that the cost increases sharply as soon as the first free variable is introduced. A natural remedy, used by concolic and dynamic-symbolic tools [4, 24], is to explore one path at a time, giving the solver only the branch conditions along that path. For RuSMT, this suggests a symbolic walker over the DSL's syntax tree that maintains a path condition and forks at each conditional.

The synthesis alphabet stops at U+2FFFF. The Unicode Strings theory of Z3 is defined over the code points U+0000–U+2FFFF [31], so the backend bounds every synthesized character to $[\text{U+0000}, \text{U+D7FF}] \cup [\text{U+E000}, \text{U+2FFFF}]$, i.e., that alphabet intersected with the scalar values that Rust can hold. This costs completeness and not soundness; a witness we accept is still a witness, but a divergence that only appears above U+2FFFF is not found.

9 Conclusion

RuSMT demonstrates a practical approach to generating conformance tests from executable specifications. For TOML 1.1.0, the result is a generated suite covering 146 of 183 markers. Running it against four production parsers found 12 inputs on which the parsers and the oracle did not all agree, and among them a defect in `smol-toml` that had not been reported before: an array-of-tables header closed by a single bracket is accepted. The other divergences are two defects that the maintainers already track and 9 inputs that test behaviour the specification leaves open.

The evaluation also exposes a limitation of whole-program synthesis. Asking Z3 to generate a complete input by inverting the lowered semantics works well when the resulting formula remains small, but becomes impractical as the symbolic execution grows. The language model therefore serves as a proposer when direct synthesis fails, while the solver remains responsible for deciding whether each proposed input satisfies the original query. A next step could be to avoid lowering the entire semantics into a single query and instead generate constraints along individual execution paths.

Data-Availability Statement

The framework, the two executable specifications, the generated suites, the differential-testing harness, and the recorded run behind every number in this paper are archived at Zenodo [10]. The emitted SMT-LIB queries are not archived, since they can be regenerated deterministically from the specifications.

References

[1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-Guided Synthesis. In *2013 Formal Methods in Computer-Aided Design (FMCAD 2013)*. IEEE, 1–8. doi:10.1109/FMCAD.2013.6679385

[2] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. Springer, 193–207. doi:10.1007/3-540-49059-0_14

[3] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 114–129. doi:10.1109/SP.2014.15

[4] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI 2008)*. USENIX Association, 209–224. https://www.usenix.org/conference/osdi-08/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems

[5] Edmund Clarke, Daniel Kroening, and Flavio Lerda. 2004. A Tool for Checking ANSI-C Programs. In *TACAS*. 168–176. doi:10.1007/978-3-540-24730-2_15

[6] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008) (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24

[7] Ecma International TC39. 2026. Test262: ECMAScript Conformance Test Suite. https://github.com/tc39/test262. Accessed 2026-08-25.

[8] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. 2009. *Semantics Engineering with PLT Redex*. MIT Press. ISBN 9780262062756.

[9] Haryadi S. Gunawi, Cindy Rubio-González, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Ben Liblit. 2008. EIO: Error Handling is Occasionally Correct. In *6th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 207–222. https://www.usenix.org/legacy/events/fast08/tech/gunawi.html

[10] Mehrad Haghshenas and Meng Xu. 2026. Reproduction Package for Article 'RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer'. Zenodo. doi:10.5281/zenodo.22095547

[11] Claude Jard and Thierry Jéron. 2005. TGV: Theory, Principles and Algorithms. *Int. J. Softw. Tools Technol. Transf.* 7, 4 (2005), 297–315. doi:10.1007/s10009-004-0153-x

[12] Sumit Kumar Jha, Susmit Jha, Patrick Lincoln, Nathaniel D. Bastian, Alvaro Velasquez, Rickard Ewetz, and Sandeep Neema. 2023. Counterexample Guided Inductive Synthesis Using Large Language Models and Satisfiability Solving. In *IEEE Military Communications Conference (MILCOM 2023)*. IEEE. doi:10.1109/MILCOM58377.2023.10356332

[13] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd Millstein, and George Varghese. 2022. SCALE: Automatically Finding RFC Compliance Bugs in DNS Nameservers. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2022)*. USENIX Association, 307–323. https://www.usenix.org/conference/nsdi22/presentation/kakarla

[14] Jinwoo Kim, Qinheping Hu, Loris D'Antoni, and Thomas W. Reps. 2021. Semantics-Guided Synthesis. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–32. doi:10.1145/3434311

[15] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th International Conference on Software Engineering (ICSE 2023)*. IEEE. doi:10.1109/ICSE48619.2023.00085

[16] Yihe Li, Ruijie Meng, and Gregory J. Duck. 2025. Large Language Model Powered Symbolic Execution. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 385 (2025). doi:10.1145/3763163

[17] Jiangyi Liu, Charlie Murphy, Anvay Grover, Keith J. C. Johnson, Thomas W. Reps, and Loris D'Antoni. 2024. Synthesizing Formal Semantics from Executable Interpreters. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024). doi:10.1145/3689724 Introduces the Synantic tool.

[18] Kunhang Lv, Yuhang Dong, Rui Han, Fuqi Jia, Feifei Ma, and Jian Zhang. 2026. LLM-Guided Quantified SMT Solving over Uninterpreted Functions. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 17 (2026), 14304–14312. doi:10.1609/aaai.v40i17.38445 Introduces the AquaForte framework.

[19] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2024. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. arXiv:2401.08807. doi:10.48550/arXiv.2401.08807

[20] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.

[21] Dominic P. Mulligan, Scott Owens, Kathryn E. Gray, Tom Ridge, and Peter Sewell. 2014. Lem: Reusable Engineering of Real-World Semantics. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*. ACM, 175–188. doi:10.1145/2628136.2628143

[22] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. 2021. JEST: N+1-version Differential Testing of Both JavaScript Engines and Specification. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE 2021)*. IEEE, 13–24. doi:10.1109/ICSE43902.2021.00015

[23] Jihyeok Park, Dongjun Youn, Kanguk Lee, and Sukyoung Ryu. 2023. Feature-Sensitive Coverage for Conformance Testing of Programming Language Implementations. *Proceedings of the ACM on Programming Languages* 7, PLDI, Article 126 (2023), 23 pages. doi:10.1145/3591240

[24] Sebastian Poeplau and Aurélien Francillon. 2020. Symbolic Execution with SymCC: Don't Interpret, Compile!. In *USENIX Security*. 181–198. https://www.usenix.org/conference/usenixsecurity20/presentation/poeplau

[25] Tom Preston-Werner et al. 2025. TOML: Tom's Obvious, Minimal Language, Version 1.1.0. https://toml.io/en/v1.1.0. Language specification.

[26] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434. doi:10.1016/j.jlap.2010.03.012

[27] Len Sassaman, Meredith L. Patterson, Sergey Bratus, and Michael E. Locasto. 2013. Security Applications of Formal Language Theory. *IEEE Systems Journal* 7, 3 (2013), 489–500. doi:10.1109/JSYST.2012.2222000

[28] Nicolas Seriot. 2016. Parsing JSON is a Minefield. http://seriot.ch/parsing_json.php. Accessed 2026-06-20.

[29] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Tom Ridge, Susmit Sarkar, and Rok Strniša. 2007. Ott: Effective Tool Support for the Working Semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*. ACM, 1–12. doi:10.1145/1291151.1291155

[30] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2006)*. ACM, 404–415. doi:10.1145/1168857.1168907

[31] Cesare Tinelli, Clark Barrett, and Pascal Fontaine. 2020. SMT-LIB Theory of Unicode Strings. https://smt-lib.org/theories-UnicodeStrings.shtml. Accessed 2026-08-24.

[32] Emina Torlak and Rastislav Bodík. 2013. Growing Solver-Aided Languages with Rosette. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward! 2013)*. ACM, 135–152. doi:10.1145/2509578.2509586

[33] Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2014)*. ACM, 530–541. doi:10.1145/2594291.2594340

[34] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhme. 2025. Cottontail: Large Language Model-Driven Concolic Execution for Highly Structured Test Input Generation. *arXiv preprint arXiv:2504.17542* (2025). arXiv:2504.17542 doi:10.48550/arXiv.2504.17542 To appear, IEEE Symposium on Security and Privacy (S&P) 2026.

[35] Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. 2024. Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification. In *Computer Aided Verification (CAV 2024) (Lecture Notes in Computer Science)*. Springer. doi:10.1007/978-3-031-65630-9_16 arXiv:2404.00762.

[36] Glynn Winskel. 1993. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press. ISBN 9780262231695. doi:10.7551/mitpress/3054.001.0001

[37] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *International Conference on Learning Representations (ICLR 2024)*. arXiv:2310.04870 https://proceedings.iclr.cc/paper_files/paper/2024/hash/0c86142265c5e2c900613dd1d031cb90-Abstract-Conference.html

[38] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2011)*. ACM, 283–294. doi:10.1145/1993498.1993532

[39] Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao, Yongle Zhang, Pranay U. Jain, and Michael Stumm. 2014. Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 249–265. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/yuan

Received 2026-07-01; accepted 2026-07-16